Índice de Contenidos

1. Resumen e introducción 2

1.1. Breve presentación del proyecto y su finalidad 2

1.2. Justificación de la elección del tema 2

1.3. Descripción general del sistema 2

2. Planteamiento del problema o necesidad 3

2.1. Situación y contexto 3

2.2. Problema detectado 3

2.3. Público objetivo 4

3. Objetivos del proyecto 4

3.1. Objetivo general 4

3.2. Objetivos específicos 5

4. Análisis previo y estudio de viabilidad X

4.1. Requisitos funcionales y no funcionales X

4.2. Tecnologías, lenguajes y herramientas seleccionadas X

4.3. Justificación de las decisiones técnicas X

4.4. Viabilidad técnica y temporal del proyecto X

5. Diseño de la aplicación X

5.1. Arquitectura del sistema X

5.2. Diagramas UML X

5.3. Diseño de la interfaz de usuario X

1. Resumen e Introducción

1.1. Breve presentación del proyecto y su finalidad

Con este proyecto damos vida a **LudiGest**, una aplicación de escritorio multiplataforma diseñada para poner orden y digitalizar el día a día operativo de las asociaciones y clubes de juegos de mesa. El propósito principal de este software es centralizar toda la información que estas comunidades manejan a diario, posibilitando la transición desde sistemas manuales, hojas de cálculo compartidas o archivos dispersos hacia una plataforma unificada, segura y fácil de utilizar.

1.2. Justificación de la elección del tema

El sector de los juegos de mesa ha experimentado un crecimiento exponencial en los últimos años, lo que se traduce en más asociaciones, más socios y ludotecas cada vez más grandes. Sin embargo, hemos constatado que la administración interna de estas agrupaciones sufre un notable estancamiento tecnológico. Se invierten incontables horas en tareas repetitivas de inventario o buscando información técnica de los juegos.

Elegimos este tema porque resuelve una necesidad real y palpable que hemos vivido de primera mano en asociaciones (como es el caso de LexLudi). Además, a nivel académico, nos permite aplicar de forma práctica, integrada y coherente todos los conocimientos adquiridos durante el ciclo de DAM: diseño de interfaces de usuario (UI), creación de **APIs RESTful**, diseño de bases de datos relacionales y consumo de servicios web externos.

1.3. Descripción general de la aplicación

LudiGest se aleja de la arquitectura de las aplicaciones monolíticas tradicionales. Hemos apostado por una arquitectura moderna cliente-servidor dividida en capas lógicas. Por un lado, tenemos una interfaz gráfica (Frontend) desarrollada en **Java** y **JavaFX**, donde hemos aplicado un diseño visual moderno y responsivo mediante el uso de hojas de estilo **CSS** globales.

Esta interfaz no ataca a la base de datos de forma directa (lo cual supondría una grave brecha de seguridad), sino que se comunica mediante peticiones **HTTP** con nuestro propio Backend, desarrollado en **Spring Boot**. Este servidor intermedio es el encargado de procesar la lógica de negocio, interactuar con la base de datos **MySQL** y, como característica estrella del proyecto, conectarse a la **API** pública internacional de **BoardGameGeek** (BGG) para extraer y automatizar la descarga de toda la información técnica de los juegos de mesa.

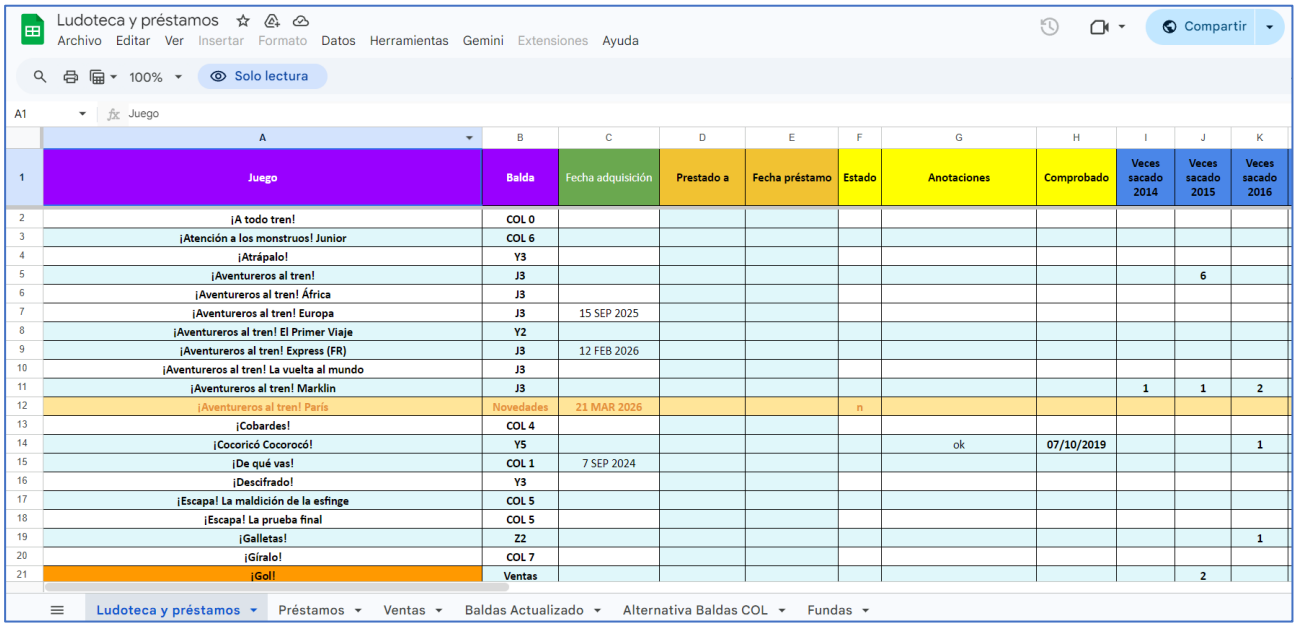
2. Planteamiento del problema o necesidad

2.1. Situación o contexto que motiva el desarrollo

Como se ha comentado, las asociaciones lúdicas modernas manejan colecciones que a menudo superan los cientos o miles de referencias de juegos. Tienen un ecosistema muy vivo, con un flujo constante de altas de socios, compras de nuevo material, préstamos y organización de eventos. Es una logística compleja que requiere un control riguroso para no colapsar.

2.2. Problema o necesidad detectada

El gran problema que hemos detectado es que la **gestión** actual es puramente "**artesanal**". Lo habitual en este nicho es encontrarnos con inventarios llevados en hojas de cálculo compartidas (**Google Sheets**) que se desactualizan rápido, foros web anticuados o peticiones de socios que se pierden en el ruido de los grupos de **WhatsApp**.



	A	B	C	D	E	F	G	H	I	J	K
1	Juego	Balda	Fecha adquisición	Prestado a	Fecha préstamo	Estado	Anotaciones	Comprobado	Veces sacado 2014	Veces sacado 2015	Veces sacado 2016
2	¡A todo tren!	COL 0									
3	¡Atención a los monstruos! Junior	COL 6									
4	¡Atrápalo!	Y3									
5	¡Aventureros al tren!	J3								6	
6	¡Aventureros al tren! África	J3									
7	¡Aventureros al tren! Europa	J3	15 SEP 2025								
8	¡Aventureros al tren! El Primer Viaje	Y2									
9	¡Aventureros al tren! Express (FR)	J3	12 FEB 2026								
10	¡Aventureros al tren! La vuelta al mundo	J3									
11	¡Aventureros al tren! Marklin	J3							1	1	2
12	¡Aventureros al tren! París	Novedades	21 MAR 2026			n					
13	¡Cobardes!	COL 4									
14	¡Cocoricó Cocorocó!	Y5					ok	07/10/2019			1
15	¡De qué vas!	COL 1	7 SEP 2024								
16	¡Descifrado!	Y3									
17	¡Escapa! La maldición de la esfinge	COL 5									
18	¡Escapa! La prueba final	COL 5									
19	¡Galletas!	Z2									1
20	¡Gíralo!	COL 7									
21	¡Gol!	Ventas								2	

Imagen 1 Ejemplo de gestión actual mediante hojas de cálculo susceptibles a errores humanos, duplicidades y desorganización estructural.

A esta desorganización se suma un problema técnico importante: dar de alta un juego de mesa nuevo en una base de datos de forma manual es una labor ingente y propensa al **error humano**. Hay que teclear el título original, los autores, la duración, el peso (dificultad), las mecánicas e incrustar la imagen de la portada. Esto provoca que los inventarios de las asociaciones siempre estén incompletos o contengan datos erróneos. Era imprescindible desarrollar una herramienta que no solo persistiera los datos, sino que hiciera el "trabajo sucio" de data entry por los administradores.

2.3. Público objetivo

La arquitectura y la interfaz de la aplicación están pensadas para satisfacer a dos perfiles principales:

Administradores / Junta Directiva: Necesitan una herramienta robusta para dar de alta juegos de forma automatizada, auditar el estado físico del inventario (juegos perdidos, dañados) y tener una radiografía exacta de qué hay en las estanterías de su local.

Socios (Usuarios finales): Buscan un catálogo visual, rápido y atractivo donde puedan filtrar dinámicamente a qué quieren jugar hoy (por número de jugadores, dificultad, duración, etc.) sin tener que rebuscar físicamente en los armarios de la asociación.

3. Objetivos del proyecto

3.1. Objetivo general

El objetivo principal del TFG es desarrollar una **plataforma de escritorio escalable y robusta** que automatice la gestión del catálogo físico e inventario de una asociación de juegos de mesa, utilizando una arquitectura orientada a microservicios (**API REST**) que desacople claramente la capa de presentación visual de la lógica de procesamiento de datos.

3.2. Objetivos específicos

Para alcanzar esta meta general, nos propusimos cumplir los siguientes hitos técnicos y funcionales:

Centralización e Integridad de Datos: Diseñar y desplegar una base de datos relacional en **MySQL**, aplicando las reglas de normalización pertinentes para garantizar la integridad de los datos físicos de la ludoteca (los ejemplares reales).

Desarrollo de API propia: Construir una **API REST** en el framework **Spring Boot** que actúe como escudo de seguridad (backend) y gestor lógico entre la base de datos y la interfaz de usuario.

Automatización y Web Scraping (Integración BGG): Integrar el servicio **XML/JSON** de la **API** pública de **BoardGameGeek** en nuestro servidor para permitir la importación automática y masiva de metadatos técnicos, mecánicas e imágenes en alta resolución con solo introducir el identificador del juego, ahorrando drásticamente horas de trabajo manual.

Interfaz de Usuario Avanzada: Crear un entorno gráfico intuitivo en **JavaFX** aplicando el patrón de diseño **Master-Detail**. El objetivo es abandonar las interfaces grises nativas de Java mediante la implementación de un motor de temas **CSS** globales (Vibrant Dark).

Filtrado Reactivo en tiempo real: Implementar un motor de búsqueda y filtrado en el Frontend que permita al usuario cruzar múltiples variables en memoria (duración, número de jugadores, dificultad y estado físico) de manera instantánea, sin sobrecargar la base de datos MySQL con peticiones SQL constantes.

Fundamentos para la Escalabilidad: Dejar la arquitectura base preparada a nivel de entidades **JPA** (con relaciones OneToMany y llaves foráneas bien estructuradas) para futuras expansiones del software, permitiendo la fácil integración posterior de un módulo de préstamos a socios o la gestión de torneos y eventos.

4. Análisis previo y estudio de viabilidad

Para no caer en el error de principiante de empezar a programar sin un rumbo arquitectónico claro, dedicamos esta fase inicial a definir con exactitud qué debía hacer el sistema, cómo debía comportarse bajo estrés y qué stack tecnológico nos ofrecía mayores garantías.

4.1. Requisitos funcionales y no funcionales

A. Requisitos Funcionales (El alcance operativo):

Gestión de Catálogo Híbrida: El sistema debe distinguir semánticamente entre la "Ficha Técnica" de un juego (metadatos teóricos invariables como autor, duración o mecánicas) y el "Ejemplar Físico" (la caja tangible en la estantería, con su código interno, estado de conservación y ubicación).

Importación Externa (Automatización): El administrador debe poder buscar e importar juegos directamente desde la **API** de **BoardGameGeek**, autocompletando los campos complejos para anular el registro manual y sus consecuentes errores tipográficos.

Filtrado Combinado Dinámico: Los socios deben poder filtrar el catálogo en tiempo real cruzando múltiples parámetros (número de jugadores, dureza, estado físico y texto libre).

Control de Accesos (RBAC): El sistema debe implementar un Control de Acceso Basado en Roles, mostrando u ocultando interfaces de gestión dependiendo de si el usuario logueado es **Socio** estándar **o Administrador**.

B. Requisitos No Funcionales (Métricas de calidad y comportamiento):

Rendimiento y Reactividad: Las búsquedas en el catálogo deben ser percibidas como instantáneas. Por ello, exigimos que el **filtrado** múltiple se realice **en** la memoria del **cliente** (RAM) y no saturando la base de datos con peticiones SQL por cada letra tecleada.

Seguridad e Integridad: El cliente de escritorio (Frontend) jamás debe contener credenciales directas a la base de datos. Toda transacción debe pasar por la capa de **validación** de nuestra **API REST**.

Usabilidad Visual (UX/UI): La interfaz debe abandonar los componentes grises por defecto de Java. Exigimos un motor de temas **CSS** que ofrezca alto contraste, jerarquía visual y una experiencia moderna.

Escalabilidad: El **backend** debe devolver las **respuestas** en un formato estándar (**JSON**), permitiendo que el día de mañana se pueda desarrollar una aplicación móvil nativa consumiendo exactamente la misma lógica de servidor.

4.2. Tecnologías, lenguajes y herramientas seleccionadas

Para cubrir estas exigentes métricas, somos consecuentes y proponemos el **stack tecnológico que hemos estado usando durante los dos años del grado**, que además es profesional y plenamente alineado con la demanda del mercado laboral actual:

Backend (Lógica y API REST): **Java 21 LTS** orquestado por el framework **Spring Boot 4.0.5**. Utilizamos Spring Data **JPA** (**Hibernate**) como **ORM** para mapear nuestras clases Java a las tablas relacionales de forma transparente.

Frontend (Cliente Visual): **JavaFX 21** gestionado mediante **Maven**. Para la construcción del árbol de vistas empleamos **Scene Builder**, lo que nos permitió aplicar el **patrón MVC**, separando drásticamente el código de marcado visual (FXML) de los controladores lógicos en Java.

Base de Datos: **MySQL 8**. Elegimos este robusto motor relacional por su fiabilidad en la gestión de llaves foráneas y restricciones de integridad (imprescindible para vincular los juegos de referencia con sus múltiples ejemplares físicos).

Servicios Externos: **API XML2** oficial de **BoardGameGeek**.

4.3. Justificación de las decisiones técnicas

Durante el diseño de la arquitectura tomamos decisiones clave que definen la calidad del software. La más crítica fue **separar físicamente el Frontend del Backend**. Hubiera sido mucho más rápido desarrollar una aplicación monolítica (donde JavaFX atacara directamente a MySQL mediante JDBC), pero decidimos montar un **servidor intermedio (API REST)**. Esto nos otorga una capa de seguridad infranqueable y nos ha permitido enfrentarnos a los retos reales de la arquitectura cliente-servidor distribuida.

Otra decisión vital fue implementar el **patrón de Heavy Scraping delegado**. En lugar de que la aplicación visual de JavaFX llamara directamente a la BGG, decidimos que fuera el servidor **Spring Boot** quien **lidiara con la API externa**, descargara los pesados XML, los parseara, los limpiara y le sirviera a nuestro Frontend un objeto JSON "masticado" y ultraligero. Así evitamos bloqueos de IP por parte de la BGG, centralizamos el procesamiento y reducimos drásticamente el consumo de red en el ordenador del usuario final.

4.4. Viabilidad técnica y temporal del proyecto

Viabilidad Técnica: Consideramos que era un proyecto viable y seguro, ya que las herramientas elegidas (**Java, Spring, MySQL**) conforman el núcleo duro formativo del ciclo de DAM. Disponíamos de la base teórica; el reto residía en la orquestación e integración de todas ellas.

Viabilidad Temporal: Este fue, sin duda, el mayor **desafío**. En los prototipos iniciales pecamos de ambiciosos, planteando la integración de módulos de torneos y terminales táctiles de autoservicio. Tras un sano ejercicio de realismo, hicimos un **recorte funcional estricto** (evitando el temido Scope Creep). Decidimos **centrar todo nuestro tiempo y esfuerzo** en dejar el núcleo principal **("Catálogo - Automatización BGG - Base de Datos")** impecable, robusto y libre de bugs, en lugar de entregar muchas funcionalidades a medio terminar. Gracias a esta decisión de gestión de proyecto, la entrega en los plazos marcados fue un rotundo éxito.

5. Diseño de la aplicación

5.1. Arquitectura del sistema

LudiGest se sostiene sobre una **arquitectura** estricta de **tres capas** desacopladas:

Capa de Presentación (Frontend): Sigue el **patrón MVC** adaptado a **JavaFX**. Las vistas (.**fxml**) actúan como lienzos "tontos" que solo exponen datos, mientras que sus Controladores reaccionan a los eventos del usuario, instanciando hilos de ejecución asíncronos para comunicarse con el servidor sin congelar la interfaz gráfica.

Capa de Negocio (Backend): Nuestro servidor **Spring Boot** actúa como un "**Proxy**" inteligente. Controla la autenticación, valida que no existan códigos de ejemplares duplicados y aplica lógica de negocio compleja (como calcular de forma dinámica la antigüedad de una compra para etiquetarla visualmente como "Novedad").

Capa de Datos: La base de datos relacional **MySQL**, normalizada en Tercera Forma Normal (**3FN**).

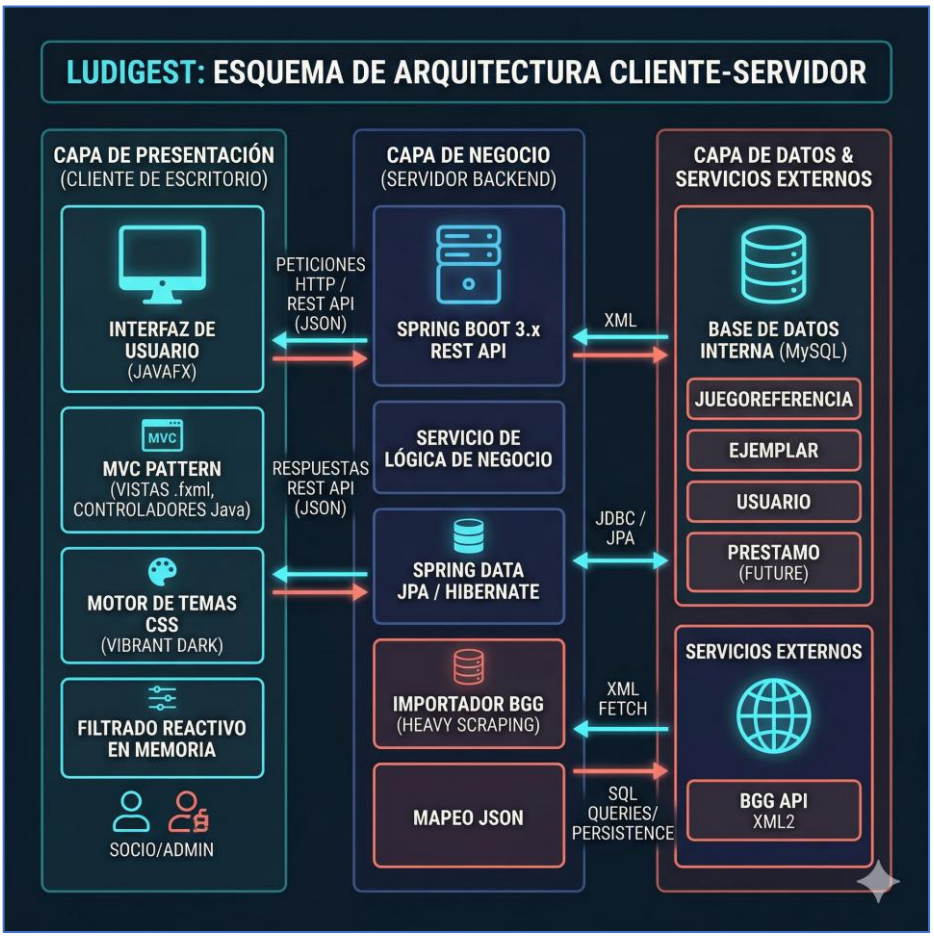


Imagen 2 - Esquema de la arquitectura cliente-servidor de LudiGest, mostrando la separación de la capa de presentación (JavaFX) de la lógica de negocio (Spring Boot) y orígenes de datos.

5.2. Diagramas UML

Para modelar la estructura del software con antelación, desarrollamos diagramas que sirvieron como "planos arquitectónicos" del sistema:

Diagrama de Casos de Uso: Refleja la interacción entre los actores (Socio y Administrador) y el sistema, destacando cómo el Administrador hereda los permisos base pero desbloquea módulos críticos como el alta híbrida de juegos (Ver Imagen 3 en Anexo I).

Diagrama de Clases: Muestra el corazón relacional de la base de datos. La clave maestra de este diseño es la relación estructural de Uno a Muchos (**1:N**) entre la entidad **JuegoReferencia** (el concepto abstracto importado de la BGG) y la entidad **Ejemplar** (la copia física inventariada) (Ver Imagen 4 en Anexo I).

Diagrama de Secuencia (Importación BGG): Detalla minuciosamente el flujo de comunicación asíncrona y la estricta delegación de responsabilidades en nuestra arquitectura. Ilustra cómo el *Frontend* se limita a solicitar la información, siendo el *Backend* (Spring Boot) quien asume la verdadera carga computacional pesada: establece la conexión con la API externa, parsea el complejo árbol XML de respuesta y persiste los datos en la base de datos, antes de devolver un objeto JSON limpio y estructurado al cliente *(Ver Imagen 5, Anexo I)*.

5.3. Diseño de la interfaz de usuario (UX/UI)

Nuestra premisa fue huir radicalmente de la típica aplicación de gestión con aspecto de "hoja de cálculo gris". El objetivo era construir un **diseño moderno, inmersivo y adaptado** a la estética contemporánea del nicho de los juegos de mesa.

En lugar de incrustar estilos pantalla por pantalla, diseñamos una arquitectura **CSS** global. Creamos un tema personalizado (fondos en tonos profundos, combinados con colores neón vibrantes para los botones de acción) que se **inyecta** directamente **a nivel de Scene**, garantizando una coherencia visual total.

La **pantalla principal** del catálogo se concibió bajo el **patrón Master-Detail:**

A la izquierda (Master), una rejilla infinita (**FlowPane**) genera dinámicamente "tarjetas" visuales con las portadas en alta calidad de los juegos.

A la derecha (Detail), un panel dinámico (**StackPane**) expone la ficha técnica completa o el formulario de edición al hacer clic en una tarjeta.

Adicionalmente, incorporamos un motor visual de **etiquetas (Badges)** de estado con psicología de color (**Verde: Disponible, Rojo: Prestado, Morado: Cuarentena**), logrando que los administradores comprendan la situación global de la ludoteca con un simple vistazo rápido. (Nota: Capturas detalladas de la interfaz final se recogen en el Anexo II).

ANEXO I

Diagramas UML

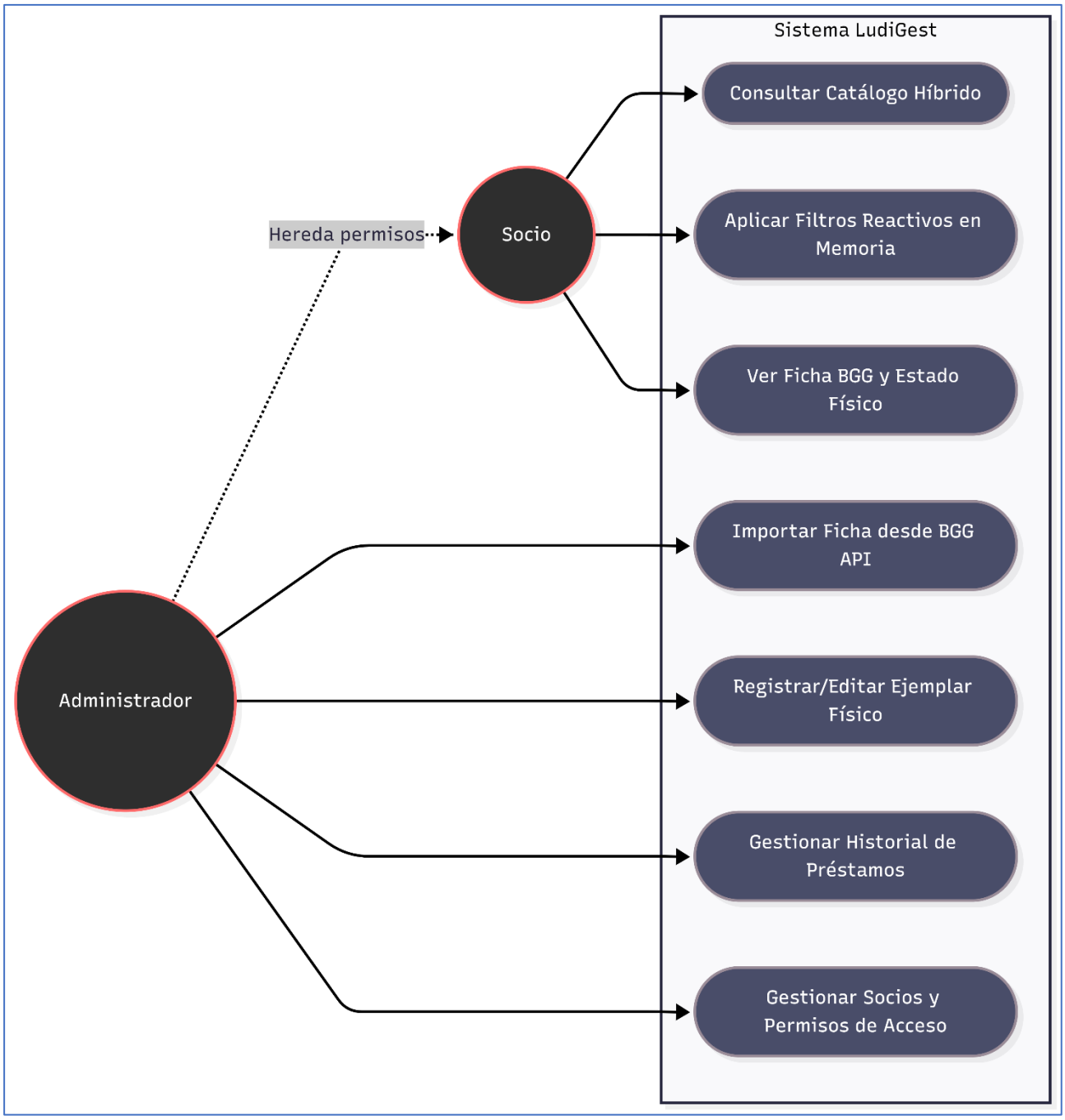


Imagen 3 - Diagrama de Casos de Uso del sistema LudiGest. Representa la interacción de los dos perfiles principales (Socio y Administrador). Se ilustra la herencia de permisos y el acceso exclusivo del Administrador a módulos críticos como la importación de datos desde BoardGameGeek (BGG), el control físico de los ejemplares y la gestión de préstamos.

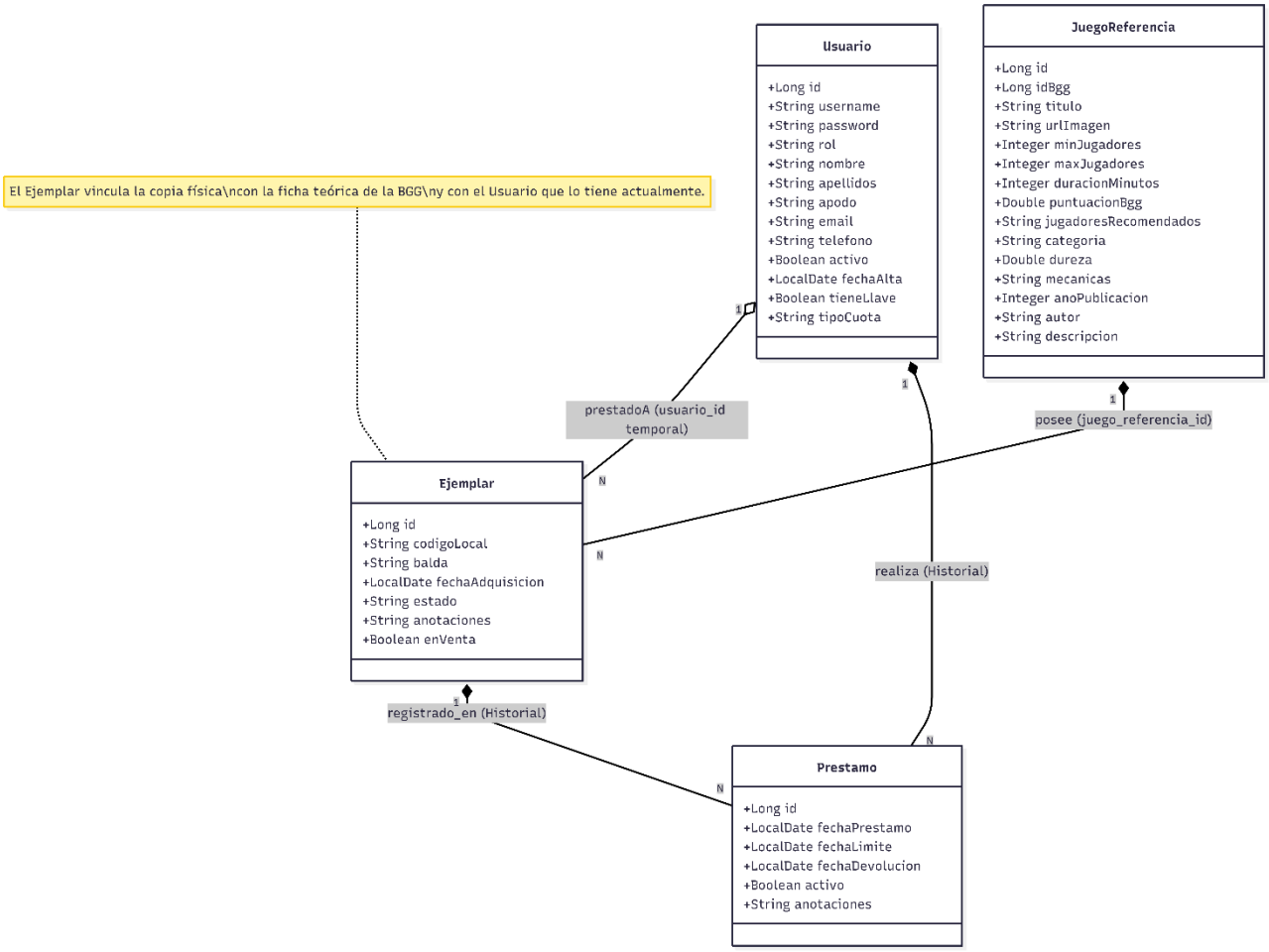


Imagen 4 - Diagrama de clases del modelo de datos principal (Backend). Refleja la separación estructural clave entre la información teórica importada de la API (JuegoReferencia) y la gestión del inventario físico del local (Ejemplar). Además, detalla las entidades de control de Usuarios y la trazabilidad relacional del historial de Préstamos.

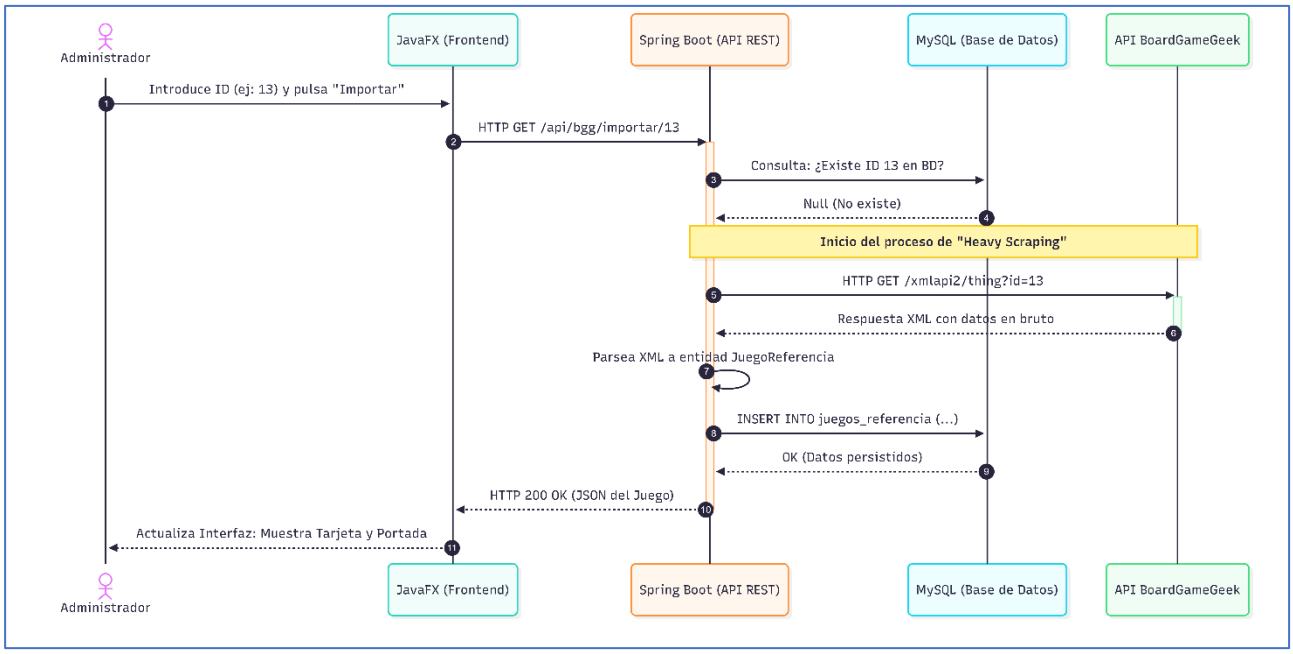


Imagen 5 - Diagrama de Secuencia del proceso de importación BGG. Se evidencia la aplicación práctica del principio de Separación de Responsabilidades (Separation of Concerns): el cliente JavaFX se mantiene ultraligero, delegando la latencia de red (conexión a la BGG) y la transformación masiva de datos (parseo del XML a entidad relacional) al servidor Spring Boot.

f